AN UNIFIED AI-BASED NETWORK INTRUSION DETECTION FRAMEWORK

Architecture Document

High-Level Design (HLD) & Detailed Design (DD)

Department of Networking and Communications | SRM Institute of Science and Technology | ICISCN 2026

---

1. Introduction

This Architecture Document describes the High-Level Design (HLD) and Detailed Design (DD) of the AI-IDS-SOC (Artificial Intelligence – Intrusion Detection System – Security Operations Center) framework. The system is a unified, end-to-end cybersecurity pipeline that combines machine learning-based network flow analysis with host-level command monitoring, all surfaced through a real-time SOC dashboard.

The document is structured as follows: Section 2 presents the system overview and design goals. Section 3 covers the High-Level Design with the overall architecture. Section 4 provides the Detailed Design of each component. Section 5 describes the data flow. Section 6 covers technology stack and deployment.

2. Design Goals & Constraints

Design Goal	Description
Real-Time Detection	Process and classify network flows and CLI commands with minimal latency
High Detection Rate	Achieve TPR >= 95% and FPR <= 3% across combined ensemble models
Modularity	Each model (IF, RF, GNN) operates independently and can be updated/retrain
Scalability	Support small-to-medium SOC environments; extensible to enterprise scale
Low Resource Footprint	Run on standard Linux hardware without GPUs for the core pipeline
Open Source	Built entirely with open-source tools: Python, Flask, SQLite, PyTorch Geometric

Constraint	Detail
Dataset Size	Validated on 200 labeled flows (113 attacks, 87 benign) — extendable
Storage	SQLite for current version; PostgreSQL migration planned for scale
OS	Linux-based deployment (Ubuntu 20.04+)
Language	Python 3.x with scikit-learn, PyTorch Geometric, Scapy, Flask

3. High-Level Design (HLD)

The system is divided into four major processing layers that work in sequence. The diagram below shows the high-level architecture:

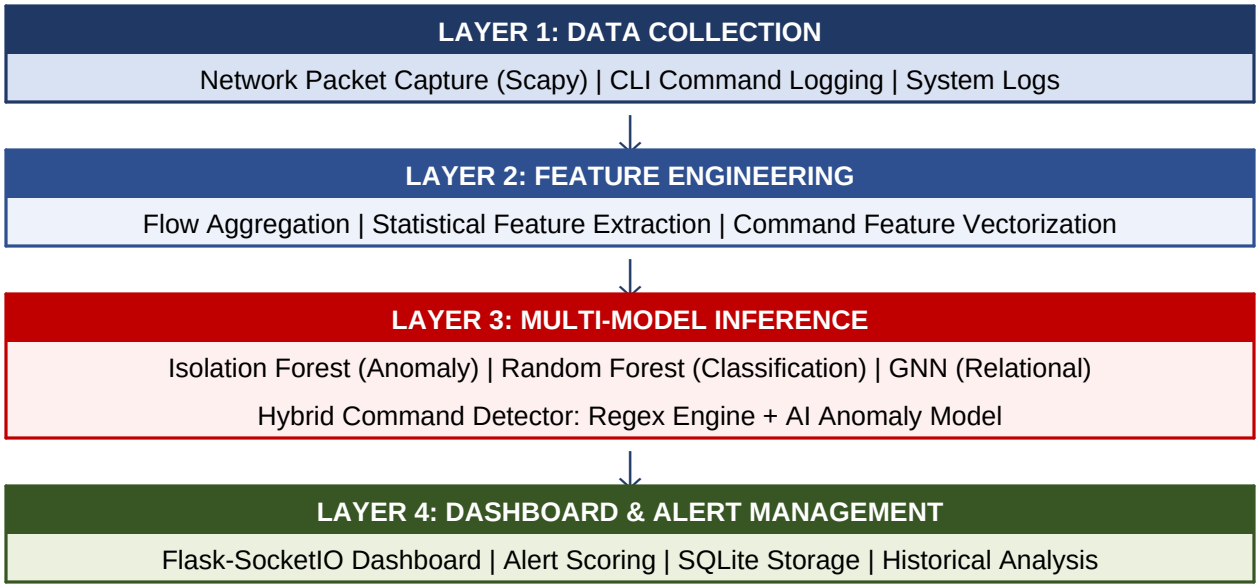

Figure 1: High-Level Architecture of AI-IDS-SOC System

3.1 System Components Overview

Component	Type	Role
Scapy Packet Capture	Data Collection	Captures raw network packets from monitored interfaces
Flow Aggregator	Preprocessing	Groups packets into flows with statistical features
Isolation Forest	ML Model	Unsupervised anomaly scoring for zero-day detection
Random Forest	ML Model	Supervised classification of known attack types
Graph Neural Network	ML Model	Structural/relational pattern detection across flows
Regex Engine	Rule Engine	47 predefined patterns for known malicious commands
Command Anomaly Model	ML Model	AI detection of novel/unknown command-level attacks
SQLite Database	Storage	Persists alerts, flow records, and model outputs
Flask-SocketIO Server	Backend API	Serves REST API and WebSocket events to dashboard
Web Dashboard	Frontend	Real-time SOC visualization for analysts

4. Detailed Design (DD)

4.1 Network Layer Security

The network layer performs end-to-end flow-based threat detection using three complementary AI models applied in cascade.

■ **Step 1: Packet Capture & Flow Construction**

Scapy monitors the network interface in promiscuous mode.

Packets are grouped into bidirectional flows using a 5-tuple key: (src_ip, dst_ip, src_port, dst_port, protocol).

Each flow is converted to a feature vector with the following attributes:

Feature	Description
packet_count	Total packets in the flow
byte_count	Total bytes transferred
fwd_packet_count	Forward-direction packet count
bwd_packet_count	Backward-direction packet count
flow_duration	Duration of the flow in seconds
mean_packet_size	Average packet size (bytes)
protocol	TCP / UDP / ICMP encoded as integer
flag_distribution	Counts of SYN, ACK, FIN, RST flags

■ **Step 2: Isolation Forest – Anomaly Scoring**

Model Type: Unsupervised (no labels required)

Assigns an anomaly score to each flow; flows below threshold flagged as suspicious.

Particularly effective for zero-day and previously unseen attack patterns.

Output: Anomaly Score (float, -1 = anomaly, 1 = normal)

■ **Step 3: Random Forest – Flow Classification**

Model Type: Supervised (trained on labeled flow dataset)

Classifies each flow into: Benign, Port Scan, C2 Beacon, Data Exfiltration.

Achieved 95.6% TPR and 2.6% FPR on 200 labeled flows.

Output: Class Label + Confidence Score

■ **Step 4: Graph Neural Network – Relational Detection**

Model Type: Graph-based (PyTorch Geometric)

Builds a dynamic k-NN graph across a sliding window of flows:

Nodes = individual flows; Edges = k nearest neighbors by Euclidean distance in feature space.

Architecture: 2x GCNConv layers → ReLU + Dropout(0.2) → Sigmoid output (0–1 probability).

Detects multi-stage and lateral movement attacks spanning multiple flows.

Output: Maliciousness Probability per flow (0.0 – 1.0)

4.2 Host Layer Security – Hybrid Command Detector

The host layer monitors CLI activity using a two-path hybrid detection system.

Path	Method	Detects	Output
Path A (Regex)	47 predefined Regex patterns	Known: reverse shells, Base64 payloads, LOTL tools	CRITICAL / HIGH / MEDIUM severity
Path B (AI)	Isolation Forest on command features	Novel/unknown malicious commands	Anomaly score + AI confidence
Verdict Fusion	Hybrid logic (Algorithm 1)	Combined confidence verdict	Malicious / Benign + Confidence

Command features extracted for AI model:

Command length, Shannon entropy, token count, special character frequency, base64 likelihood score.

4.3 Alert Management & Storage

All detections are scored and stored in SQLite with the following schema:

Table	Key Fields	Purpose
alerts	id, timestamp, type, severity, confidence, source_ip, dst_ip	Stores all network and command alerts
flows	id, timestamp, features_json, if_score, rf_label, gnn_prob	Flow records with model outputs
commands	id, timestamp, command_str, regex_match, AI_score, verdict	CLI command audit log with verdicts

4.4 Flask-SocketIO Dashboard

The real-time dashboard is the primary interface for SOC analysts.

Dashboard Panel	Content
Live Alerts Feed	Real-time WebSocket stream of incoming alerts with severity badges
Network Flow Graph	Visual representation of GNN flow graph with attack nodes highlighted
Model Metrics	Live TPR, FPR, and confidence score gauges for each model
Alert History	Searchable, filterable table of historical alerts from SQLite
Command Monitor	Live CLI activity log with verdict (Safe / Suspicious / Malicious)
System Status	Health indicators for each model and data pipeline component

5. Data Flow Diagram

The following table describes the complete end-to-end data flow through the system:

Step	Input	Process	Output
1	Raw network packets	Scapy capture on NIC	Packet stream
2	Packet stream	Flow aggregation (5-tuple grouping)	Flow feature vectors
3	Flow feature vectors	Isolation Forest scoring	Anomaly scores
4	Flow feature vectors	Random Forest classification	Attack labels + confidence
5	Flow feature vectors	GNN graph construction + inference	Maliciousness probability
6	All model outputs	Ensemble fusion + alert scoring	Final alert with severity
7	Final alert	SQLite storage	Persisted alert record
8	Persisted alert	Flask-SocketIO WebSocket push	Live dashboard update
9	CLI command string	Regex pattern matching (47 rules)	Pattern match + severity
10	CLI command string	Feature extraction + IF anomaly scoring	AI confidence score
11	Regex + AI outputs	Hybrid verdict fusion (Algorithm 1)	Final command verdict

6. Technology Stack & Deployment

Category	Technology	Version / Notes
Packet Capture	Scapy	Python library for raw packet capture and manipulation
ML Framework	scikit-learn	Isolation Forest, Random Forest implementation
Graph ML	PyTorch Geometric	GCNConv layers for GNN model
Backend API	Flask + Flask-SocketIO	REST API + WebSocket server
Database	SQLite	Lightweight alert and flow storage (PostgreSQL for scale)
Frontend	HTML / JS / Chart.js	Real-time SOC dashboard UI
OS	Ubuntu 20.04+	Linux-based deployment environment
Language	Python 3.10+	Primary implementation language
Regex Engine	Python re module	47 handcrafted patterns for command detection

7. Performance Summary

Model	Accuracy	Precision	Recall (TPR)	F1-Score
Random Forest	78.4%	92.1%	79.9%	85.6%
Isolation Forest	73.3%	88.4%	65.4%	75.2%
Graph Neural Network	80.5%	87.3%	90.1%	88.7%
Ensemble Model	89.5%	94.7%	94.5%	94.6%

Model	TPR (%)	FPR (%)	Threat Coverage

Random Forest	95.6	2.6	Known attack signatures (Port Scan, C2, Exfiltration)
Isolation Forest	65.5	2.7	Zero-day / Novel behavioral anomalies
GNN	58.2	1.1	Multi-stage / Structural / Lateral movement patterns
Ensemble Model	97.2	1.2	Comprehensive – all threat categories combined